

Детализированное описание репозитория prog-6-ml-service

Анализ программного кода, конфигурации .gitlab-ci.yml, docker-compose-dev.yml и
инфраструктурных файлов
Репозиторий: gitlab.herzen.spb.ru/1148905/prog-6-ml-service

Краткое резюме

Репозиторий содержит ML-сервис для предсказания одобрения ипотеки/займа. Решение состоит из FastAPI backend с моделью машинного обучения, статического frontend под Nginx, локального docker-compose окружения, GitLab CI/CD и Ansible-сценариев для деплоя на сервер.

Анализ выполнен по доступным публичным страницам GitLab, README, истории коммитов и видимым фрагментам кода/диффов. В местах, где raw-файлы GitLab UI отдавал частично, выводы сформулированы по доступным фрагментам и отмечены как потенциальные замечания.

Параметр	Описание
Тип проекта	ML web service: FastAPI backend + статический frontend под Nginx
ML-задача	Предсказание loan approval / mortgage approval по анкетным и финансовым признакам
Локальный запуск	docker compose -f docker-compose-dev.yml up --build
CI/CD	GitLab CI: lint backend, build/push Docker images, Ansible configure/deploy
Инфраструктура	Docker, Nginx, Ansible, GitLab Container Registry, nginx-proxy + acme companion
Ключевые риски	Несоответствие API-префиксов, небезопасная загрузка pkl, мягкий CI, потенциальные ошибки frontend JS

Содержание

1. Назначение проекта
2. Общая структура репозитория
3. Backend: FastAPI ML-сервис
4. Модели данных backend/src/models.py
5. Frontend
6. docker-compose-dev.yml
7. Dockerfile'ы и Nginx
8. .gitlab-ci.yml
9. Ansible/CD
10. Ключевые замечания по качеству и рискам
11. Итоговое описание
12. Источники

Примечание: номера источников в квадратных скобках соответствуют разделу «Источники» в конце отчета.

1. Назначение проекта

Репозиторий prog-6-ml-service содержит ML-сервис для предсказания одобрения ипотеки/займа. По доступному описанию проекта и README, локальный запуск выполняется командой `docker compose -f docker-compose-dev.yml up --build`, а frontend должен подхватывать изменения из директории сборки без полного пересоздания образа. [1], [3]

Функционально приложение решает задачу приема пользовательских признаков, применения заранее подготовленного препроцессора и ML-модели, возврата результата предсказания и вероятности, а также обработки CSV-файлов для пакетного предсказания.

Возможность	Где реализовано	Смысл
Проверка доступности	/ping, /status	Health/status endpoint'ы
Загрузка модели	/upload-model	Загрузка .pkl модели и попытка ее активировать
JSON-предсказание	/predict	Один заявитель или список заявителей
CSV-предсказание	/predict-from-csv	Batch prediction; при наличии фактических меток считается ROC-AUC
Валидация входных данных	backend/src/models.py	Pydantic-модели, Enum-типы и схемы ответа

2. Общая структура репозитория

По истории коммитов и видимым файлам проект включает код backend, frontend, Docker-конфигурации, GitLab CI/CD и Ansible-инфраструктуру. Крупный инфраструктурный коммит добавил .gitlab-ci.yml, README, Ansible-поли, docker-compose-dev.yml, Dockerfile'ы frontend и Nginx-конфигурацию. [3]

```

.
├── README.md
├── .gitignore
├── .gitlab-ci.yml
├── docker-compose-dev.yml
├── backend/
│   ├── Dockerfile
│   ├── requirements.txt
│   ├── .dockerignore
│   ├── assets/
│   │   ├── best_model.pkl
│   │   ├── preprocessor.pkl
│   │   └── tmp_model.pkl
│   └── src/
│       ├── main.py
│       └── models.py
├── frontend/
│   ├── Dockerfile
│   ├── Dockerfile.dev
│   ├── .dockerignore
│   ├── nginx.conf
│   └── build/
│       ├── index.html
│       └── style.css
└── ansible/
    ├── inventory.yml
    ├── site-playbook.yml
    ├── group_vars/all.yml
    └── roles/
        ├── common/
        └── deploy_server/

```

3. Backend: FastAPI ML-сервис

Backend реализован на FastAPI. В коде он описан как “Machine Learning Prediction Service” для решений loan approval. Сервис предоставляет endpoint’ы для проверки состояния, загрузки модели, одиночных и пакетных предсказаний, а также обработки CSV-файлов с возможным расчетом ROC-AUC. [4]

3.1. Основные endpoint’ы

Endpoint	Описание
GET /ping	Возвращает строку из переменной окружения PONG_RESPONSE, по умолчанию “pong”. Используется как простая проверка живости сервиса.
GET /status	Возвращает status=“up” и значение DEPLOYED_FROM. В видимом коде не возвращает признаки загрузки модели/препроцессора.
POST /upload-model	Принимает UploadFile, сохраняет во временный путь ./assets/tmp_model.pkl, проверяет через joblib.load, перемещает в MODEL_PATH и вызывает load_artifacts().
POST /predict	Принимает PersonRequest или список PersonRequest; при загруженных MODEL и PREPROCESSOR возвращает предсказания PersonResponse.
POST /predict-from-csv	Принимает CSV, ожидает набор колонок признаков и опционально loan_status; при наличии меток рассчитывает ROC-AUC.

3.2. Загрузка ML-артефактов

Backend ожидает два основных файла: препроцессор и обученную модель. Функция загрузки проверяет существование обоих файлов и затем загружает их через `joblib.load`. [4]

```

./assets/preprocessor.pkl
./assets/best_model.pkl

```

Такой подход соответствует типичной sklearn-архитектуре: входные признаки сначала преобразуются препроцессором, затем передаются модели. При этом загрузка pickle/joblib-файлов от пользователя требует осторожности, так как десериализация недоверенных объектов может быть опасной.

4. Модели данных backend/src/models.py

models.py содержит Pydantic-модели и Enum-типы для валидации API. Модуль задает схемы запросов/ответов и документацию для API. [4]

Enum	Значения
Gender	female, male
Education	Associate, Bachelor, Doctorate, High School, Master
HomeOwnership	MORTGAGE, OTHER, OWN, RENT
LoanIntent	DEBTCONSOLIDATION, EDUCATION, HOMEIMPROVEMENT, MEDICAL, PERSONAL, VENTURE
PreviousLoanDefaultsOnFile	No, Yes

PersonRequest включает возраст, пол, образование, доход, опыт работы, тип владения жильем, сумму кредита, цель кредита, ставку, отношение кредита к доходу, длину кредитной истории, кредитный рейтинг и наличие прошлых дефолтов. PersonResponse наследует PersonRequest и добавляет loan_status и loan_proba. PredictionResponse содержит список предсказаний и опциональный roc_auc. [4]

5. Frontend

Frontend представляет собой статическую HTML/CSS/JS-страницу, раздаваемую через Nginx. В начальной версии интерфейс содержал кнопку Ping API, которая выполняла fetch('/api/ping'). [3]

Позже интерфейс был расширен до страницы "ML Сервис Ипотеки" с проверкой модели на сервере, загрузкой .pkl, переходом к форме предсказания и пакетным CSV-предсказанием. Видимые элементы UI включают "Проверить модель на сервере", "Загрузить модель", "Перейти к предсказанию" и "Пакетное предсказание (CSV)". [5]

Nginx-конфигурация frontend проксирует запросы /api/ на backend-сервис по имени контейнера backend:80, а статические файлы раздает из /usr/share/nginx/html. Поэтому внешний API доступен через префикс /api, хотя внутри FastAPI endpoint'ы объявлены без этого префикса. [3], [4]

6. docker-compose-dev.yml

docker-compose-dev.yml предназначен для локальной разработки и поднимает два сервиса: frontend и backend. Frontend монтирует локальную директорию сборки в Nginx-контейнер и публикует порт 8000, backend запускает Uvicorn на внутреннем порту 80. [3]

6.1. Frontend-сервис

```
frontend:
  build:
    context: ./frontend/
    dockerfile: Dockerfile.dev
  container_name: ml-service-frontend
  restart: unless-stopped
  volumes:
    - ./frontend/build:/usr/share/nginx/html:ro
  ports:
    - "8000:80"
  depends_on:
    - backend
```

6.2. Backend-сервис

```
backend:
  build:
    context: ./backend/
    dockerfile: Dockerfile
  command: uvicorn main:app --host 0.0.0.0 --port 80 --reload
  container_name: ml-service-backend
  restart: unless-stopped
  environment:
    PONG_RESPONSE: "pong from backend!"
```

Backend не публикует порт наружу напрямую: к нему обращается frontend/Nginx по внутренней Docker-сети. Такая схема близка к production-подходу, где frontend выполняет роль reverse проху для API.

7. Dockerfile'ы и Nginx

Backend Dockerfile был переработан в multi-stage-стиле: используется `python:3.14-slim` как builder, зависимости ставятся из `requirements.txt`, затем во второй slim-образ копируются установленные пакеты, исходный код `src` и ML-артефакты `assets`. [6]

Также добавлен Docker HEALTHCHECK, который обращается к `http://127.0.0.1:80/api/status`. Это место выглядит проблемным: backend-код объявляет `/status`, а не `/api/status`; префикс `/api` появляется только на уровне Nginx-проксирования. Поэтому healthcheck внутри backend-контейнера, вероятно, должен обращаться к `/status`. [4], [6]

Frontend Dockerfile и Dockerfile.dev основаны на `nginx:stable-alpine`, копируют `nginx.conf` в `/etc/nginx/conf.d/default.conf`, а содержимое `build/` - в `/usr/share/nginx/html/`. [3]

8. .gitlab-ci.yml

CI/CD построен вокруг Docker-in-Docker, сборки backend/frontend образов и деплоя через Ansible. В ранней версии `.gitlab-ci.yml` был общий anchor для Docker build/push: login в registry, сборка образа, push тега `$IMAGE_TAG`, затем `tag/push latest`. [3]

Для Ansible используется подготовительный anchor: создается `~/.ssh`, приватный ключ берется из переменной `CD_SLAVE_PRIVKEY`, декодируется из base64, выставляются права и устанавливаются Ansible Galaxy-зависимости. [3]

Job	Назначение
Lint backend	Запускает <code>ruff check --output-format=gitlab .</code>
Build and push backend image	Собирает и публикует backend Docker image
Build and push frontend image	Собирает и публикует frontend Docker image
Configure servers	Запускает Ansible с теом <code>packages_slave</code>

Job	Назначение
Deploy to servers	Запускает Ansible с тегом <code>deploy_slave</code>

В CI есть несколько спорных моментов: `allow_failure: true` на `build/deploy` снижает строгость pipeline; в одном из фрагментов `frontend build-job` видна самозависимость в `needs`; `stage configure_servers` содержит опечатку; `lint_frontend` закомментирован. [4], [6], [7]

9. Ansible/CD

Ansible-часть предназначена для подготовки сервера и деплоя Docker Compose-приложения. `group_vars/all.yml` содержит адрес `registry`, имена `backend/frontend` образов, теги по умолчанию, директорию приложения, домен, email для Let's Encrypt и переменные для логина в `registry`. `inventory.yml` содержит хост `slave1` с IP-адресом 178.72.170.34 и пользователем `root`. [3]

Роль `common` обновляет пакеты, ставит Docker и `docker compose plugin`, добавляет Docker apt repository и запускает Docker service. Роль `deploy_server` создает директорию приложения, копирует `docker-compose` шаблон, логинится в `registry`, останавливает старые сервисы и запускает новые через `community.docker.docker_compose_v2`. [3]

Сервис production-шаблона	Назначение
frontend	Образ frontend, virtual host и Let's Encrypt env
backend	Образ backend, env <code>PONG_RESPONSE</code> , позднее <code>DEPLOYED_FROM</code>
nginx-proxy	Reverse proxy <code>jwilder/nginx-proxy</code>
nginx-proxy-acme	Автоматическое получение TLS-сертификатов

Production-шаблон открывает порты 80 и 443, монтирует Docker socket и директории для сертификатов, `vhost`, `html` и `acme`. [3]

10. Ключевые замечания по качеству и рискам

№	Замечание
1	Несоответствие <code>/api/status</code> и <code>/status</code> . Backend объявляет <code>/status</code> , но Docker healthcheck обращается к <code>/api/status</code> . Внутри backend-контейнера Nginx-префикс <code>/api</code> не применяется, поэтому healthcheck может падать. [4], [6]
2	Frontend ожидает поля статуса модели, которых <code>/status</code> не возвращает. В frontend-коде проверяются <code>model_is_loaded</code> и <code>preprocessor_is_loaded</code> , а backend <code>/status</code> возвращает только <code>status</code> и <code>deployed_from</code> . [4], [5]
3	Загрузка <code>.pkl</code> через <code>joblib.load</code> небезопасна для недоверенных файлов. Endpoint <code>/upload-model</code> десериализует загруженный файл. Это допустимо только при полном доверии к пользователю или при дополнительной изоляции. [4]
4	<code>/upload-model</code> может не обновлять глобальную модель в памяти. В видимом фрагменте присваивается <code>MODEL, PREPROCESSOR = load_artifacts()</code> ; если нет <code>global MODEL, PREPROCESSOR</code> , присваивание будет локальным. [4]
5	Непоследовательный формат <code>loan_status</code> . Документация <code>PersonResponse</code> говорит об <code>approved/denied</code> , а CSV-ветка формирует 0/1 как строки. [4]
6	CI слишком мягкий для production-проекта. <code>allow_failure: true</code> на <code>build/deploy</code> позволяет не блокировать pipeline при ошибках критичных операций. [7]
7	Есть следы ручных/экспериментальных правок frontend. В последних diff'ах встречаются подозрительные JS-фрагменты вокруг <code>data.message</code> , <code>Data.message</code> и <code>datamodel_status_msg</code> ; стоит проверить локальный запуск и консоль браузера. [8]

11. Итоговое описание

prog-6-ml-service - учебный/практический ML-сервис полного цикла. Backend на FastAPI обслуживает sklearn/joblib-модель, frontend на статическом HTML/JS общается с API через Nginx reverse proxy, локальная разработка запускается через docker-compose-dev.yml, а CI/CD собирает Docker-образы и деплоит их на сервер через Ansible.

Репозиторий содержит не только код приложения, но и инфраструктуру: Dockerfile'ы, Nginx-конфиг, GitLab CI, Ansible-роли, inventory, production compose-шаблон и README-инструкции.

Сильные стороны: контейнеризация, разделение frontend/backend, Pandantic-схемы, CSV batch prediction, расчет ROC-AUC, registry-based deploy, Ansible automation и lint backend через Ruff.

Основные зоны доработки: выровнять API-префиксы /api/*, исправить /status под ожидания frontend и healthcheck, убрать небезопасную загрузку pickle без авторизации/изоляции, проверить global при обновлении модели, привести loan_status к единому формату, ужесточить CI и почистить frontend JS.

12. Источники

Ниже перечислены страницы GitLab, по которым был составлен отчет. При передаче PDF в систему без доступа к GitLab ссылки могут быть недоступны внешним пользователям.

- [1] Страница проекта prog-6-ml-service - <https://gitlab.herzen.spb.ru/1148905/prog-6-ml-service>
- [2] История коммитов ветки master - <https://gitlab.herzen.spb.ru/1148905/prog-6-ml-service/-/commits/master>
- [3] Коммит "CI/CD для бекенда", добавление CI/CD, Docker Compose, frontend, Ansible - <https://gitlab.herzen.spb.ru/1148905/prog-6-ml-service/-/commit/4f78d54191aff2d8afb4cbb8d8b4dfb8a3526bf2>
- [4] Коммит/дифф с backend FastAPI, endpoint'ами, Pydantic-моделями и lint backend - <https://gitlab.herzen.spb.ru/1148905/prog-6-ml-service/-/commit/2b682053c3e122222fc5af383889539ae1d06a56>
- [5] Коммит с расширением frontend "ML Сервис Ипотеки" - <https://gitlab.herzen.spb.ru/1148905/prog-6-ml-service/-/commit/0b612e22e315779e0ba602babb3b2c294f821ac1>
- [6] Коммит с Dockerfile/healthcheck и изменениями CI - <https://gitlab.herzen.spb.ru/1148905/prog-6-ml-service/-/commit/e6b36ef98c72699ce073131515158c9e75aceaf2>
- [7] Коммит с allow_failure для build/deploy job'ов - <https://gitlab.herzen.spb.ru/1148905/prog-6-ml-service/-/commit/cc7b0aa660468ed23943439c449c1063e9c74124>
- [8] Коммит с последними правками HTML/frontend JS - <https://gitlab.herzen.spb.ru/1148905/prog-6-ml-service/-/commit/7358327bf213081aba1cb4ef7244f95d3ebfe263>